Programming in C++

Exercise 1: Getting Started in C++ (01a_first_program.cpp):

```
#include <iostream>

void other_function() {
   std::cout << "here is another function" << std::endl;
}

int main() {
   std::cout << "Welcome to CSC 115!" << std::endl;

   return 0;
}
```

To run a C++ program named **hello.cpp**:
1) To compile: **`g++ -Wall -std=c++20 -o output hello.cpp`**
2) If there are no errors, run the executable: **`./output`**

General process: (1) Edit code. (2) Save. (3) Compile. (4) Debug. (5) Execute

Exercise 2: Variables (01b_variables.cpp):

```
int a = 5;
int b = 14;
int c = b/a;
int d = (double)b/a;

std::cout << "c: " <<  c << std::endl;
std::cout << "b/a: " <<  b/a << std::endl;
std::cout << "d: " <<  d << std::endl;
std::cout << "(double)b/a: " <<  (double)b/a << std::endl;

// Comment one line with //
// or multiple with /* <comments here> */
double x = 5.0;
double y = 14.0;
double z = y/x;

std::cout << "z (x/y): " <<  z << std::endl;

z = b / a;
std::cout << "z (b/a): " <<  z << std::endl;

z = (double)b / a;
std::cout << "z ((double)b/a): " <<  z << std::endl;
```

Exercise 3: Ifs, Loops (01c_ifs_loops.cpp):

```
#include <iostream>
int get_highest(int a, int b, int c);
void print_range(int a, int b);

int main() {
    int res = 0;
    result = get_highest(1, 2, 3);
    std::cout << "get_highest(1,2,3): " << res << std::endl;
    result = get_highest(6, 5, 4);
    std::cout << "get_highest(6,5,4): " << res << std::endl;
    result = get_highest(-5, -2, -4);
    std::cout << "get_highest(-5,-2,-4):" << res << std::endl;
    result = get_highest(9, 9, 9);
    std::cout << "get_highest(9,9,9): " << res << std::endl;
    print_range(1, 8);
    print_range(109, 84);
    return 0;
}

/**
 * @brief Returns the maximum of the three given integers
 * @param a the first integer to compare
 * @param b the second integer to compare
 * @param c the third integer to compare
 * @return the maximum value between a, b, and c
 */
int get_highest(int a, int b, int c) {
   // if-statement: if (condition) { <run if true> }









}

/**
 * @brief Prints out all values from a to b (inclusive)
 * @param a the first integer to print
 * @param b the last integer to print
 */
void print_range(int a, int b) {
    // for-loop: for(initialization; condition; update)







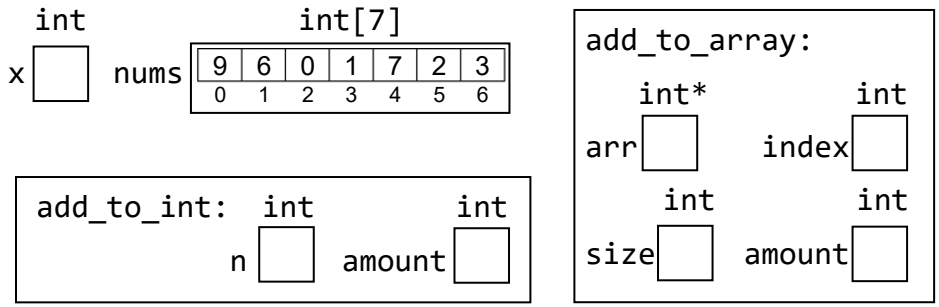


}
```

Exercise 4: Array Review (01d_arrays.cpp):
We will complete a memory trace of the following code:

```
int x{5};
std::cout << "x before add_to: " << x << std::endl;
add_to_int(x, 4);
std::cout << "x after add_to: " << x << std::endl;

int nums[]{9, 6, 0, 1, 7, 2, 3};
std::cout << "nums[1] before add_to: " << nums[1] << std::endl;
add_to_array(nums, 7, 1, 4);
std::cout << "nums[1] after add_to: " << nums[1] << std::endl;
std::cout << "nums[-1]: " << nums[-1] << std::endl;
std::cout << "nums[10]: " << nums[10] << std::endl;
```




```
/**
 * @brief adds amount to the given int, n
 * @param n the integer to add onto
 * @param amount the amount to add to n
 */
void add_to_int(int n, int amount) {
    n += amount;
}

/**
 * @brief adds amount to the value in the array at given index
 * @param arr the array of integers
 * @param size the number of elements in the array
 * @param index the index of the value to modify in the array
 * @param amount the amount to add to n
 * @pre 0 <= index < size
 */
void add_to_array(int arr[], int size, int index, int amount){
    arr[index] += amount;
}
```

Exercise 5: Programming with Arrays (01d_arrays.cpp):

```
/**
 * @brief Returns the sum of positive integers in the array
 * @param array the array of integers
 * @size the number of elements in the array
 * @return the sum of all positive integers
 */
int sum_positive(int array[], int size) {









}

/**
 * @brief Doubles the value of all elements in the array
 * @param array the array of integers
 * @size the number of elements in the array
 */
void double_all(int array[], int size) {




}

/**
 * @brief Determines if the array contains only odd values
 * @param array the array of integers
 * @size the number of elements in the array
 * @return true if no even values are present, false otherwise
 */
bool all_odd(int array[], int size) {









}
```

Exercise 6: Working with Structs (01e_structs.cpp):

```
struct City {
    std::string name;
    int population;
};

bool is_bigger(City c1, City c2);
int count_bigger(City c, City cities[], int size);
City get_biggest(City cities[], int size);

int main() {
   // Creating objects for testing:
   City vic{"Victoria", 100000};
   City tor{"Toronto", 2800000};
   City mon{"Montreal", 1775000};
   City van{"Vancouver", 675000};
   City cal{"Calgary", 1600000};
   City edm{"Winnipeg", 750000};
   City cities[]{vic, tor, mon, van, cal, edm};
   City west_cities[] {vic, van, cal};

   // is_bigger tests:
   is_bigger(vic, van); // should be false (van)
   is_bigger(tor, cal); // should be true (tor)
   is_bigger(tor, tor); // should be false (tied)
   // Notice: bool values are output as 1 (true) or 0 (false)

   // count_bigger tests:
   int result{0};
   result = count_bigger(van, cities, 6);
   std::cout << result << std::endl;
   result = count_bigger(vic, cities, 6);
   std::cout << result << std::endl;
   result = count_bigger(tor, cities, 6);
   std::cout << result << std::endl;
   result = count_bigger(van, west_cities, 3);
   std::cout << result << std::endl;

   // get_biggest tests:
   City biggest{get_biggest(cities, 6)};
   std::cout << "in cities: " << biggest.name << std::endl;
   biggest = get_biggest(west_cities, 3);
   std::cout << "west_cities: " << biggest.name << std::endl;

   return 0;
}
```

```
/**
 * @brief Determines if c1 has a larger population than c2
 * @param c1 the first city to compare
 * @param c2 the second city to compare
 * @return true if c1 has a larger population, false otherwise
 */
bool is_bigger(City c1, City c2) {

}

/**
 * @brief Returns number of cities larger than c in the array
 * @param c the city to compare against
 * @param cities the array of cities
 * @param size the number of cities in the array
 * @return number of cities in array w/ larger pop than c
 */
int count_bigger(City c, City cities[], int size) {









}

/**
 * @brief Returns the biggest city in the array
 * @param cities the array of cities
 * @param size the number of cities in the array
 * @return The city in the array with the largest population
 *         (or the city that comes earliest in array if tied)
 * @pre size > 0 (cities is not empty)
 */
City get_biggest(City cities[], int size) {









}
```